Vkv

Truncated Horvitz–Thompson estimation with certified fetching for the attention denominator

VerifiedKV algorithm explainer · 23 October 2026

At prefill, Vkv keeps each tail token with probability r_i (KVzip scores) and moves the rest to CPU. At every attention decision it does three things, all computed from the query sketch and r, never from which tokens were drawn. First, it *truncates* the tail tokens whose Horvitz–Thompson upweighting is too risky: if sampled, they get weight 1 instead of $1/r$. The set is chosen by an MSE criterion. Second, it bounds the resulting error with a Bernstein certificate that treats the known truncation bias and the random fluctuation separately. Third, if the bound exceeds the tolerance ε_0, it fetches the heaviest tokens by sketch weight, and picks how many to fetch in a single prefix-sum sweep. With exact weights in the plan, every decision then satisfies $\mathbb{P}(|\widehat{B} - B| > \varepsilon_0 B) \le \delta$. The effect of sketch error is left to Part III.

Part I Theory

1 Setting

Fix one (layer, KV head) and one decision, i.e. one query row. The context has keys k_i, and the query q (scaled by $1/\sqrt{d}$) gives weights $w_i = e^{\langle q, k_i\rangle - m}$. The quantity to estimate is the softmax denominator

$$B = \sum_i w_i + w^x,$$

where w^x is the exact weight of the question and generated keys.

Prefill. Each context token has a keep probability $r_i = \min(1, c\tau_i)$ from KVzip scores τ_i. Sinks and the local window get $r_i = 1$. Write $H = \{r_i = 1\}$ for the always-resident set and $L = \{r_i < 1\}$ for the tail. Independent draws $Z_i \sim \text{Bernoulli}(r_i)$ decide which tail tokens stay on the GPU; call these the sample $S = \{i \in L : Z_i = 1\}$. The full cache is offloaded to the CPU.

Sketch. A compact sketch of all keys gives an approximate weight $\hat{w}_i$ for every tail token, sampled or not. In the analysis below we take the plan's weights to be exact ($\hat{w}_i = w_i$). The effect of sketch error is discussed in Part III.

Assumption A1 (Draw independence)**.** The Z_i are independent, and independent of everything a plan may use: the query, the sketch, and r.

This holds in the question pass when draws are independent across layers. At decode it holds along a fixed reference prefix.

2 The base estimator and why plans must ignore the draw

The Horvitz–Thompson (HT) estimator weights each sampled tail token by $1/r_i$:

$$\widehat{B}_{\text{HT}} = \sum_H w_i + w^x + \sum_{i \in L} \frac{Z_i}{r_i} w_i, \qquad \mathbb{E}\widehat{B}_{\text{HT}} = B, \qquad \text{Var} = \sum_L w_i^2 \frac{1 - r_i}{r_i}.$$

Design rule. Every set used below (truncated, fetched) must be chosen from $(\hat{w}, r)$ only. If you instead pick sampled tokens *because* they were sampled and give them weight 1, you discard the mass they represented. On a synthetic head this biased the estimate by -25% to -41%.

3 Truncation: trading a little bias for a lot of variance

3.1 One token

Give a tail token weight 1 instead of $1/r$ if it is sampled. Its contribution changes as follows:

	weight if sampled	bias	variance
HT	$1/r$	0	$w^2(1-r)/r$
Truncated	1	$-w(1-r)$	$w^2 r(1-r)$

So truncation adds bias $b = w(1-r)$, always downward, and removes variance $w^2(1-r)/r - w^2 r(1-r) = w^2(1-r)^2(1+r)/r$.

3.2 Many tokens: why the per-token test is wrong

MSE = (total bias)^2 + total variance. Variances of independent tokens add. Biases add *before* squaring, and they all point the same way. If T is the truncated set and $\beta_T = \sum_{i \in T} w_i(1-r_i)$ its total bias, the bias term is β_T^2. Keep in mind the square of sum vs sum of square.

3.3 The MSE rule

Adding token j to T changes the MSE by

$$\Delta\text{MSE} = \underbrace{2\beta_T\, w_j(1-r_j) + w_j^2(1-r_j)^2}_{(\beta_T + b_j)^2 - \beta_T^2} - w_j^2(1-r_j)^2\frac{1+r_j}{r_j}.$$

Dividing by $w_j(1-r_j)$ gives

$$\text{truncating } j \text{ lowers the MSE} \iff J_j := \frac{w_j(1-r_j)}{r_j} > 2\beta_T.$$

J_j is token j's *jump*: how far HT overshoots when j is sampled. Go through tokens in decreasing order of jump. Jumps fall and β_T rises along that order, so the MSE first decreases and then increases, and the best set $T^\star$ is the prefix that stops at the first j with $J_j \le 2\beta_T$. Large-jump tokens (heavy weight, tiny r) are truncated first.

Lemma 1 (Truncation caps the jumps)**.** *Every tail token left as HT has $J_j \le 2\beta_{T^\star}$.*

Proof. The rule stops at the first token whose jump is at most $2\beta_{T^\star}$, and all later tokens in the order have jumps no larger. □

This is what makes the certificate in §6 tight. Without truncation, a single token with $r = 10^{-3}$ can have a jump 1000× its weight, and any Bernstein-type bound pays for that even though it happens with probability 10^{-3}.

Remark 1. **interesting thing i found:** Shrinking extreme HT weights to trade bias for variance is known in survey statistics as *weight trimming*. What is specific here is choosing the trimmed set from a query-time sketch, before the draw is seen, and using the trimming to bound the jumps so that a closed-form certificate becomes tight.

4 Fetching

Fetching a token makes it exact. For this to stay unbiased, *every* fetched token must get weight 1, not only the ones outside the sample. Tokens outside S are read from CPU; tokens already sampled lose their $1/r$ upweighting. (If sampled members kept $1/r$, a fetched token would contribute $w + w(1-r)$ on average.) A fetched token then contributes no error. The transfer cost is the number of fetched tokens that were not sampled.

Order. Fetch tail tokens in decreasing order of $\hat{w}$, as in needle fetch. Let F_k be the top k. Fetching removes a token's variance and, if it was truncated, also its bias.

5 The error after fetching k tokens

After fetching the top k tokens, every tail token falls into one of three groups. Each group contributes to the error $\widehat{B}_k - B$ in a different way.

- **Fetched** (F_k): counted with its exact weight. Error 0.
- **Truncated, not fetched** ($T^\star \setminus F_k$): counted with weight 1 if sampled, missing if dropped.
- **HT, not fetched** (the rest of L): counted with weight $1/r$ if sampled, missing if dropped.

Tokens in H and the question/generated keys are exact and contribute no error.

One truncated token. If it was sampled (probability r), we count w, which is exactly right: error 0. If it was dropped (probability $1-r$), we count nothing: error $-w$. So its error is $w(Z-1)$. On average this is $-w(1-r)$ (the bias basically), a *known* shortfall we can compute in advance. Around that average, the error moves up by $w(1-r)$ when the token is sampled, and down by wr when it is dropped.

One HT token. If it was sampled, we count w/r: error $+w(1-r)/r$ (its jump). If it was dropped, we count nothing: error $-w$. So its error is $w(Z/r-1)$. On average this is exactly 0 (this is why we call this part unbiased): the occasional overshoot cancels the frequent miss.

	if sampled	if dropped	mean	variance	max move up / down
Truncated	0	$-w$	$-w(1-r)$	$w^2 r(1-r)$	$w(1-r)$ / wr
HT	$+w(1-r)/r$	$-w$	0	$w^2(1-r)/r$	$w(1-r)/r$ / w

Adding up. Each unfetched token's error is its mean plus a random part with mean 0 (we do this so we can later apply the bernstein, which only works on mean zero vector sum). Summing over all unfetched tokens splits the total error into a known part and a random part:

$$\widehat{B}_k - B = \underbrace{-\beta(k)}_{\text{known bias}} + \underbrace{Y(k)}_{\text{random part}}, \qquad \beta(k) = \sum_{T^\star \setminus F_k} w_i(1-r_i). \tag{1}$$

- $\beta(k)$ is the sum of the truncated tokens' shortfalls. It does not depend on the draw, and it always pushes the estimate down.
- $Y(k)$ is the sum of the random parts of all unfetched tokens. They are independent (each token was kept independently) and each has expectation value zero with non zero variance. Their total variance is

  $$V(k) = \sum_{T^\star \setminus F_k} w_i^2 r_i(1-r_i) + \sum_{L \setminus (T^\star \cup F_k)} w_i^2 \frac{1-r_i}{r_i},$$

  and no single one moves away from its mean by more than $b(k)$, the largest entry of the last column over unfetched tokens.

Why this split helps. The known bias can simply be added to the error budget. Only $Y(k)$ needs a probability bound, and for that we need just $V(k)$ and $b(k)$ (next section). Fetching moves a token out of the residual groups, which removes its terms from β, V and b. That is how fetching lowers the error bound. By Lemma 1, the upward moves of the remaining HT tokens are at most $2\beta_{T^\star}$, which keeps $b(k)$ small.

6 The certificate

Lemma 2 (Bernstein)**.** *If Y is a sum of independent, zero-mean terms with $|Y_i| \le b$ and $\sum \mathrm{Var}(Y_i) = V$, then for $t > 0$, $\mathbb{P}(|Y| \ge t) \le 2\exp\big(-t^2/(2(V + bt/3))\big)$.*

Set the right-hand side equal to δ and solve. With $L_\delta = \ln(2/\delta)$,

$$t(k) = \frac{b(k)L_\delta}{3} + \sqrt{\Big(\frac{b(k)L_\delta}{3}\Big)^2 + 2V(k)L_\delta}, \qquad \boxed{\varepsilon(k) = \frac{\beta(k) + t(k)}{B}}$$

On the event $|Y(k)| \le t(k)$, which has probability at least $1 - \delta$, (1) gives $|\widehat{B}_k - B| \le \beta(k) + t(k) = \varepsilon(k)B$.

7 Decision and fetch size in one rule

Proposition 3 (Monotone along the order)**.** *$\beta(k)$, $V(k)$ and $b(k)$ never increase in k, so neither does $\varepsilon(k)$.*

Proof. Each fetch removes a non-negative term from β and from V, and a maximum over a smaller set cannot grow. t is increasing in V and b. □

Rule. If $\varepsilon(0) \le \varepsilon_0$, do not fetch. Otherwise fetch $F_{k^\star}$, where $k^\star$ is the smallest k with $\varepsilon(k) \le \varepsilon_0$. By Proposition 3, $k^\star$ is the first index that passes in one sweep. It always exists, because fetching the whole tail gives $\beta = V = b = 0$.

8 The guarantee

Theorem 4 (Vkv guarantee)**.** *Under Assumption A1, let Vkv fetch $F_{k^\star}$ with $k^\star$ the smallest k such that $\varepsilon(k) \le \varepsilon_0$ (no fetch if $k = 0$ passes). Then for every decision,*

$$\mathbb{P}\big(|\widehat{B}_{k^\star} - B| > \varepsilon_0 B\big) \le \delta.$$

Consequently the expected fraction of failing decisions is at most δ, whatever the correlation between decisions.

Proof. $T^\star$, the fetch order and $\varepsilon(k)$ depend only on the weights and r, so by Assumption A1 the chosen $k^\star$ is independent of the draw. The certificate of §6 then applies at $k^\star$. The last claim is linearity of expectation. □

Remark 2 (Marginal targets). The theorem certifies each decision separately. If transfer per forward turns out too high (one fetch set serves all query rows in a forward), replace the per-decision test by a budget on the *average* failure bound across a forward. The expected failing fraction stays bounded by linearity, and the policy can give up on a few expensive diffuse decisions.

Part II The full algorithm

Algorithm 1 runs once per context. Algorithm 2 runs for every decision, vectorised over query rows. Every quantity in lines 1–18 of Algorithm 2 is computed from $(\hat{w}, r)$ alone; the draw Z is used only to decide what to transfer and how to bias attention (lines 19–22).

Algorithm 1: Vkv prefill (once per context)

Input: context keys and values; kept fraction κ

1 Compute KVzip scores τ_i;
2 Choose c so that $\sum_i \min(1, c\tau_i) = \kappa n$; set $r_i \leftarrow \min(1, c\tau_i)$;
3 $r_i \leftarrow 1$ on sinks and the local window; $H \leftarrow \{r_i = 1\}$, $L \leftarrow \{r_i < 1\}$;
4 **foreach** *(layer, KV head)* **do**
5 Draw $Z_i \sim \text{Bernoulli}(r_i)$ independently for $i \in L$ (independent stream per (layer, KV head));
6 Keep $H \cup \{i \in L : Z_i = 1\}$ on the GPU;
7 Offload the full cache to CPU; build the key sketch; store r_i for all $i \in L$;

Algorithm 2: Vkv decision, truncation and fetch (one query row)

Input: sketch weights $\hat{w}_i$ for $i \in L$; exact w_i on H and w^x; r; tolerance ε_0; failure probability δ

Output: fetch set F; attention bias

1 $\widetilde{B} \leftarrow \sum_H w_i + w^x + \sum_L \hat{w}_i$; $\quad L_\delta \leftarrow \ln(2/\delta)$;
`// (a) truncation set by the MSE rule`
2 $J_i \leftarrow \hat{w}_i(1 - r_i)/r_i$ for $i \in L$; sort L by J in decreasing order;
3 $T^\star \leftarrow \emptyset$; $\quad \hat{\beta} \leftarrow 0$;
4 **foreach** i *in that order* **do**
5 **if** $J_i \le 2\hat{\beta}$ **then break**;
6 $T^\star \leftarrow T^\star \cup \{i\}$; $\quad \hat{\beta} \leftarrow \hat{\beta} + \hat{w}_i(1 - r_i)$;
`// (b) per-token bias, variance and maximal move`
7 **for** $i \in L$ **do**
8 **if** $i \in T^\star$ **then**
9 $b_i \leftarrow \hat{w}_i(1 - r_i)$; $\quad v_i \leftarrow \hat{w}_i^2 r_i(1 - r_i)$; $\quad m_i \leftarrow \max(\hat{w}_i(1 - r_i),\, \hat{w}_i r_i)$;
10 **else**
11 $b_i \leftarrow 0$; $\quad v_i \leftarrow \hat{w}_i^2(1 - r_i)/r_i$; $\quad m_i \leftarrow \max(\hat{w}_i(1 - r_i)/r_i,\, \hat{w}_i)$;
`// (c) one sweep in needle order,` $k = 0, 1, \ldots, |L|$
12 Sort L by $\hat{w}$ in decreasing order; let F_k be the first k tokens;
13 $\hat{\beta}(k) \leftarrow \sum_{L \setminus F_k} b_i$; $\quad \widehat{V}(k) \leftarrow \sum_{L \setminus F_k} v_i$; $\quad \hat{b}(k) \leftarrow \max_{L \setminus F_k} m_i$; `// prefix sums, suffix max`
14 $t(k) \leftarrow \dfrac{\hat{b}(k)L_\delta}{3} + \sqrt{\Big(\dfrac{\hat{b}(k)L_\delta}{3}\Big)^2 + 2\widehat{V}(k)L_\delta}$;
15 $\hat{\varepsilon}(k) \leftarrow \big(\hat{\beta}(k) + t(k)\big)/\widetilde{B}$;
`// (d) decision`
16 **if** $\hat{\varepsilon}(0) \le \varepsilon_0$ **then**
17 $F \leftarrow \emptyset$; `// no fetch`
18 **else**
19 $k^\star \leftarrow \min\{k : \hat{\varepsilon}(k) \le \varepsilon_0\}$; $\quad F \leftarrow F_{k^\star}$; $\quad$ fetch $F \setminus S$ from CPU;
`// (e) attention bias`
20 Bias 0 on $H \cup F$ and on sampled tokens in $T^\star \setminus F$;
21 Bias $\log(1/r_i)$ on sampled tokens in $L \setminus (T^\star \cup F)$;
22 Bias $-\infty$ on all other tail tokens;

The attention bias in (e) implements the estimator

$$\widehat{B} = \sum_{H} w_i + w^x + \sum_{F} w_i + \sum_{S\cap T^\star\setminus F} w_i + \sum_{S\setminus(T^\star\cup F)} \frac{w_i}{r_i},$$

where $S = \{i \in L : Z_i = 1\}$. By Proposition 3, $\hat{\varepsilon}(k)$ is non-increasing, so $k^\star$ is the first index that passes in the sweep. With exact weights ($\hat{w} = w$, so $\widetilde{B} = B$), Theorem 4 gives $\mathbb{P}(|\widehat{B} - B| > \varepsilon_0 B) \le \delta$.

Part III Implementation note

9 What the code runs (differs from the theory only where stated)

Tight certificate. The bias always points down, so bound the two sides separately and pass if

$$P_{\text{fail}}(k) = e^{-u_+^2/(2(V+b^+u_+/3))} + e^{-u_-^2/(2(V+b^-u_-/3))} \le \delta, \qquad u_\pm = \varepsilon_0\widetilde{B} \pm \beta(k),$$

with $b^\pm$ the separate up and down maxima from the table in §5, and the second term set to 1 if $u_- \le 0$. Use the same sweep and take the first passing k.

Why this is safe

If the simple check passes at k, then $t \le u_- \le u_+$, so each term is at most $\delta/2$. The tight check therefore passes wherever the simple one does: the guarantee carries over, and $k_{\text{practice}} \le k_{\text{theory}}$. What is lost is *monotonicity*. As β shrinks, u_+ shrinks too, so the over-estimation term can grow: P_{fail} can pass at k and fail at $k+1$ (112 of 20k rows in simulation). The first passing index is still the minimal passing one, but the tight check cannot be binary-searched, and "fetching more is always safe" does not hold for it. Both properties hold for the simple check only.

Sketch error (theory deferred). With real sketches, $\hat{\beta}$, $\widehat{V}$, $\hat{b}$ and $\widetilde{B}$ are estimates, so the certificate is approximate. First run with *true* weights in place of $\hat{w}$ (the simulation has them) to separate algorithm behaviour from sketch error. A simple correction to try later: inflate V by γ^2, b and β by γ, and divide $\widetilde{B}$ by γ, with γ calibrated so these hold on most decisions.

Numerics and cost.

- Two sorts, a few cumsums and one reverse cummax per row. A top-K partial sort (e.g. $K = 4096$) suffices; treat the rest as HT.
- Compute $\hat{w}^2(1-r)/r$ in fp32, since it overflows fp16 when r is tiny.
- $T^\star$ lives in the per-row attention bias, so truncation costs no transfer.
- Keep $T^\star$ frozen after fetching. Re-truncating could only lower the bound; freezing may over-fetch slightly but stays valid.
- If Bernstein is still loose, try exact Chernoff at a few ladder points rather than in the sweep. In the synthetic test it was only about 1.3× tighter.

10 What to log, and what "logical" looks like

Per decision, log: phase, layer, head; $\varepsilon(0)$; decision; $k^\star$; transfers $|F \setminus S|$; $|T^\star|$; $\beta/\widetilde{B}$ at $k^\star$; b^+ before and after truncation. Using the full cache, also log the true error $e = |\widehat{B}_{k^\star} - B|/B$, and the error the no-fetch estimator would have had.

Check	Expected if Vkv is behaving
Fetch rate vs ε_0, δ, kept fraction	Rises as each decreases; monotone curves
$\mathbb{P}(e > \varepsilon_0)$ over all decisions	$\leq \delta$; with true weights, a violation is a bug
$k^\star$ vs oracle k (smallest k with true error $\leq \varepsilon_0$)	Ratio ≥ 1; its size is the bound's slack
Tight vs simple certificate	Tight never fetches more per row; log rows where P_{fail} passes at k and fails at $k+1$
b^+ before vs after truncation	After $\leq 2\beta$ (Lemma 1); before can be orders of magnitude larger
Truncation on vs off ($T^\star = \emptyset$)	Truncation cuts fetches at matched failure rate
Transfer per forward (union over rows)	Compare with needle fetch at matched memory; if far higher, switch to the averaged budget (Remark 2)
$\beta/\widetilde{B}$ at $k^\star$	Small (a few %); watch whether it compounds downstream

Red flags.

- Failure rate above δ with true weights: a sampled set leaked into $T^\star$ or the order, or sampled members of F kept weight $1/r$.
- $T^\star$ equal to the whole tail: the bias is being summed over sampled tokens only. It must include unsampled truncated tokens.
- Fetch rate near 100%: fp16 overflow in the variance, or a sketch that overestimates weights.